

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
Organización de Lenguajes y Compiladores 2
Segundo semestre 2026
Proyecto 1



Proyecto 1:

Contacto

3xtrat3rr3str3D





Objetivos

General

- ☐ Generar código de tres direcciones a partir de un lenguaje de alto nivel como parte del proceso de compilación.

Específicos

- ☐ Validar el lenguaje de alto nivel, tanto de manera léxica, sintáctica y semántica antes de iniciar el proceso de compilación.
- ☐ Generar las estructuras adecuadas para tener una representación de cuartetos para el proceso de compilación.
- ☐ Utilizar las cuartetos generadas para generar un código funcional en el lenguaje de programación C.
- ☐ Poner en práctica los conceptos de stack y heap en el proceso de compilación.



Descripción de la actividad

Fecha: ?? de ?????? del 2026

Lugar: Escondite secreto en el salón 1

Hace un par de semanas, lograste camuflar el código mediante tu exitosa traducción a PigLatin, ahora podemos seguir programando con la seguridad de que los cerdos no interfieran en nuestros planes. El siguiente paso es contactar a la civilización extraterrestre de los Zetarianos, famosos por destruir imperios tiránicos a base de rayos láser y álgebra lineal, para esto has modificado los APs del salón 1 y te preparas para el primer contacto... 3, 2, 1...

```
t0 = "No lo entendemos"
t1 = "Zetariano"
t2 = call verificar_semantica, t0, t1
if_false t2 goto L1
print "OK"
goto L0
L1:
print "ERROR"
L0:
halt
```

>> Un desastre total, el mensaje se perdió... los zetarianos no podrán enviarnos ayuda si no nos entienden, de momento no podemos seguir arriesgarnos a que los cerdos nos encuentren...

Te escabulles a la biblioteca y encuentras el códice de "El Dragón", investigando un poco llegas a la conclusión de que los satélites alienígenas solo entienden un lenguaje universal de bajo nivel: Código de Tres Direcciones (C3D).



Tu misión, es tomar los nuevos lenguajes de alto nivel de la Resistencia, más sofisticado que el Latín, validar su semántica y traducirlo a instrucciones C3D puras para disparar la baliza de auxilio hacia las estrellas antes de que los cerdos detecten la frecuencia. ¿Podrás hacerlo?, contamos contigo cadete...

Descripción de la aplicación

En el proyecto se va a validar varios lenguajes de alto nivel, para posteriormente descomponerlos en código de tres direcciones mediante cuartetos, para validar de que estas cuartetos puedan continuar con el proceso de compilación, se deberán utilizar las estructuras generadas para tener un bytecode preliminar (básicamente código de tres direcciones ejecutable).



Descripción de los lenguajes

Y?

Este lenguaje está obsoleto para los extraterrestres, pero se rumorea de que sigue la línea de un animal terrestre llamado Pitón, o en inglés python...

Este lenguaje es case sensitive.

En este solo se pueden definir estructuras y funciones. No se pueden definir variables globales.

Las tabulaciones en este lenguaje son las más importantes. Pues son las que definen el orden del código.

Formato base: estructuras y funciones

A continuación se presenta la estructura general de un archivo en lenguaje Y?, este tiene extensión .y

```
// definición de estructuras globales, la sección es opcional
%estructuras
estructura MiEstructura:
    cadena nombre

estructura Persona:
    entero edad
    cadena nombre
    flotante promedio // numero con decimales
```



```
caracter letra
/*
    la expresión para definir arreglos
    obligatoriamente debe ser constante
    unicamente dentro de la definición de una estructura
*/
entero miArray[10]

// es posible anidar estructuras
MiEstructura miEstructura

// definición de un funciones, la seccion es obligatoria
%funciones
/*
    Importante
    Todos los tipos de datos primitivos se pasan por valor
    Los arreglos se pasan asi:
        [] entero miArray
        ejemplo: definir miFuncion([] entero miArray):
        Los arreglos se van a pasar exclusivamente por referencia
        Es decir que la función recibe un acceso directo
        a la ubicación original del arreglo en la memoria.
        Los arreglos a bajo nivel deben estar aplanados.
    Las estructuras se pasan asi:
        {} MiEstructura miEstructura
        ejemplo:
        Al igual que los arreglos,
        se pasan exclusivamente por referencia
*/
// Función sin retorno, con un parámetro. Los parametros son opcionales
definir funcionSinRetorno(entero miEntero):
    miEntero = 90 * 10

// Función con retorno de tipo entero
definir funcionConRetorno(entero miEntero) -> entero :
```



```
miEntero = 10 + 10  
retornar 160
```

Expresiones

Nota: Para las validaciones de tipos se debe tomar en cuenta la validación implícita de tipos.

Las operaciones permitidas son las siguientes:

Operador	Símbolos/palabras	Descripción
Aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Estándar universal
Relacionales	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code>	Estándar universal
Lógicos	<code>&&</code> <code> </code>	And y Not
Negación	<code>!</code>	Negación universal
Suma/Resta abreviada	<code>++</code> <code>--</code>	Suma/Resta una unidad

Uso de arreglos

```
entero numeros[5] = {10, 20, 30, 40, 50}  
entero resultado  
resultado = numeros[0] + numeros[1] // 10 + 20 = 30  
numeros[2] = numeros[0] * 3 // numeros[2] ahora vale 30
```

Uso de estructuras

```
// Se pueden declarar estructuras dentro de las funciones
```



```
estructura Puntos:
```

```
    entero x
```

```
    entero y
```

```
    flotante promedio
```

```
Punto p1 = {10, 20, 85.5}
```

```
Punto p2 = {5, 15, 90.0}
```

```
flotante sumaPromedios
```

```
entero distanciaCuadrada
```

```
sumaPromedios = p1.promedio + p2.promedio
```

```
Punto p3
```

```
p3 = p1
```

Declaración de variables

Se puede declarar una variable en cualquier parte de una función, incluido en un ciclo o estructura condicional.

```
entero sinInicializacion  
entero edadUsuario = 25  
flotante temperatura = 36.6  
caracter inicial = 'A'  
bool bandera = verdadero  
bool bandera2 = falso  
cadena saludos = "Saludos zetarianos"  
entero numeros[10]  
entero matriz[3][3]  
Persona alumno1  
alumno1.nombre = "Yennifer"
```



Estructuras condicionales

```
si(edad > 18) entonces
    imprimir("Codigo si es mayor de edad")

    si(condicion == verdadero) entonces
        imprimir("Otra condicion")

    imprimir("Esto siempre se imprime")
sino (edad == 18) entonces // opcional
    imprimir("Codigo si tiene exactamente 18")
contrario // opcional
    imprimir("Codigo si es menor de edad")
```

```
elegir(opcion) :
    caso 1:
        // Código para la opción 1
        x = 10
        romper
    caso 2:
        // Código para la opción 2
        x = 20
        romper
    siempre:
        // Código por defecto
        x = 30
        romper
```

Ciclos

```
para(entero i = 0; i < 10; i++):
    si(i == 3) entonces
```



```
    continuar // Salta esta iteración cuando i es 3
}
si(i == 8) entonces
    romper    // Sale del ciclo por completo cuando i es 8
}
```

```
entero contador = 0
mientras(contador < 5) hacer
    contador++;

    si(contador == 2) entonces
        continuar; // Salta el resto del ciclo y vuelve a evaluar
```

```
entero intentos = 0
hacer:
    intentos++

    si(intentos == 4) entonces
        romper // Rompe el ciclo inmediatamente

mientras(intentos < 10)
```

Funciones especiales

```
imprimir("Imprimir")
leer()
cadena x = leer()
x = leer()
```



Zetariano

Este lenguaje está orientado a objetos, se rumorea que esta basando en java. La extensión de este archivo es .z

El archivo tiene que llamarse como la clase definida en el.

Importante: Esta parte del proyecto tiene que estar pensada para escalar, ya que en el proyecto 2 se retomaran estos elementos.

```
public class Persona {  
    // Atributos públicos (el encapsulamiento se verá en la Parte 2)  
    String nombre;  
    int edad;  
  
    // CONSTRUCTOR: Sirve para inicializar el objeto al crearlo  
    public Persona(String nombreParametro, int edadParametro) {  
        nombre = nombreParametro;  
        edad = edadParametro;  
    }  
  
    /* Sobrecarga de constructor,  
    tambien se pueden sobrecargar metodos. */  
    public Persona() {  
        nombre = "Sin nombre";  
        edad = 0;  
    }  
  
    // MÉTODOS: Acciones o comportamientos que puede realizar el objeto  
    public void saludar() {  
        println("¡Hola! Me llamo " + nombre + " y tengo " + edad + "  
años.");  
    }  
  
    public int calcularAnioNacimiento(int anioActual) {
```



```
        return anioActual - edad;
    }
}
```

Nota 1: Los objetos deben vivir en el heap

Nota 2: el encapsulamiento, la herencia y el polimorfismo se dejará pendiente para este proyecto, no se contemplan.

Nota 3: Un flujo de objetos bien definido, por naturaleza soporta recursividad, es lo esperado.

Expresiones

Aritméticas

```
int a = 10;
int b = 3;
int suma = a + b;           // 13
int resta = a - b;          // 7
int multiplicacion = a * b; // 30
int division = a / b;        // 3 (división entera)
int modulo = a % b;          // 1 (residuo de la división)
```

Incremento y decremento

```
a++; // Incrementa en 1 (a pasa a ser 11)
b--; // Decrementa en 1 (b pasa a ser 2)
```

Lógicas

```
boolean esMayor = (a > b);           // true
boolean esIgual = (a == 10);          // depende del valor actual de a
boolean condicionAnd = (a > 5 && b < 10); // true si ambas se cumplen
```



```
boolean condicionOr = (a > 20 || b > 0); // true si al menos una se cumple
boolean condicionNot = !(a == 10);      // inversión lógica
```

De asignación

```
int x = 5;
x += 3; // Equivalente a x = x + 3 (x ahora es 8)
x -= 2; // Equivalente a x = x - 2 (x ahora es 6)
x *= 2; // Equivalente a x = x * 2 (x ahora es 12)
```

Ternario

```
int edad = 20;
String mensaje = (edad >= 18) ? "Es mayor de edad" : "Es menor de edad";
int numero = 7;
int esParOImpar = (numero % 2 == 0) ? 0 : 1; // 1 porque 7 es impar
```

Uso de arreglos

```
int[] numeros = {10, 20, 30, 40, 50};
int resultado;
resultado = numeros[0] + numeros[1]; // 10 + 20 = 30
numeros[2] = numeros[0] * 3; // numeros[2] ahora vale 30
```

Uso de objetos

```
// Suponiendo que tenemos una clase 'Persona' con atributos públicos
(nombre, edad)
Persona p1 = new Persona("Carlos", 25);
Persona p2 = new Persona("Ana", 30);
int sumaEdades;
String mensajeClasificacion;

sumaEdades = p1.edad + p2.edad; // 25 + 30 = 55
mensajeClasificacion = (p1.edad >= 18) ? "Adulto" : "Menor";
```



```
int anioNac = 2026 - p1.obtenerEdad();
if(p1 == null){
    print("Es posible comparar si un objeto es nulo");
}
```

Declaración de variables

```
int edad = 25; // Números enteros
double altura = 1.75; // Números con decimales
char inicial = 'A'; // Un solo carácter
boolean esEstudiante = true; // Valores lógicos (true/false)
int[] calificaciones = new int[5];
String[] nombres = {"Carlos", "Ana", "Pedro"};
int[][] matriz = new int[3][3];

// Suponiendo que existe una clase llamada 'Persona'
Persona alumno1; // apunta a null
alumno1 = new Persona("Carlos", 20);
Persona profesor = new Persona("Ana", 45);
```

Estructuras condicionales

```
int edad = 18;
int opcion = 2;
if (edad > 18) {
    println("Es mayor de edad.");
} else if (edad == 18) {
    println("¡Justo tiene 18 años!");
} else {
    println("Es menor de edad.");
}
```



```
// --- CUANDO SOLO HAY UNA INSTRUCCIÓN ---  
// Si el bloque del if, else if o else tiene una sola línea de código,  
// las llaves {} son opcionales:  
if (edad >= 18)  
    println("Es adulto"); // Una sola instrucción sin llaves  
else  
    println("Es menor"); // Una sola instrucción sin llaves  
  
// Aunque es código sucio, el siguiente bloque es válido  
if (condicion == true)  
    if (otraCondicion == false)  
        print("hola");
```

```
switch (opcion) {  
    case 1:  
        println("Opción 1 seleccionada");  
        break; // Crucial para salir del switch, opcional  
    case 2:  
        println("Opción 2 seleccionada");  
        // No hay un break, también ejecutará la opción default  
    default:  
        println("Opción no válida");  
        break;  
}
```

Ciclos

```
for (int i = 0; i < 5; i++) {  
    println("Iteración número: " + i);  
  
    if (i == 2) {  
        continue; // Salta a la siguiente iteración cuando i es 2  
    }  
}
```



```
}

if (i == 4) {
    break;    // Rompe y sale del ciclo por completo cuando i es 4
}
}

for ( ; ; ) { // todos sus parametros son opcionales
    println("Este bucle nunca termina a menos que use un break...");
    break;
}
```

```
int contador = 0;
while (contador < 3) {
    println("Contador while: " + contador);
    contador++;

    if (contador == 1) {
        continue; // Salta el resto del ciclo y vuelve a evaluar la
condición
    }
}
```

```
int intentos = 0;
do {
    intentos++;
    println("Intento número: " + intentos);

    if (intentos == 2) {
        break; // Rompe el ciclo do-while inmediatamente
    }
} while (intentos < 5);
```



Funciones especiales

```
println("Imprimo con un salto de linea");  
print("Imprimo sin un salto de linea");  
readln(); // lee una entrada
```

Pig Latin

El lenguaje de inicio del programa será el pig latin de la misión anterior, aunque contará con algunas modificaciones. El lenguaje es case sensitive.

Ya no se permitirá definir estructuras propias en este lenguaje, se tendrán que importar necesariamente de algún archivo .y

```
##  
    Importaciones de los otros lenguajes  
##  
import carpeta.Objeto1.z  
import carpeta.Funciones.y  
  
##  
    Sección opcional de variables, puede no existir  
    En esta sección solo se definen variables, arreglos  
    o estructuras globales  
##  
  
VARIABLES>  
esto edad : numerus 20;  
esto cifrado : bool falsus;
```



```
esto comandante : textum "Estudiante X";
esto fuerza : numerus 10;
esto poder : numerus 0;

##
    Seccion de funcion principal
    Esta sección es obligatoria
##

MAIOR>
>> "Hola comandante!" ;
>> "Ingresa tu nombre por favor" ;
comandante <<
>> "Bienvenido" >> comandante ;
>> "Ingresa tu edad" ;
edad <<

si (edad >= 18) {
    cifrado = verum;
    fuerza = 12;
} finis ;

##
    la siguiente funcion tuvo que estar definida en un archivo .y
##
>> "Tu poder es: >> " calcularPoder(fuerza);
>> "La puerta esta cifrada?" >> cifrado ;

FINIS;
```



Declaraciones

Variables

```
// Declaración de entero
esto mi_entero : numerus 10;
esto total : numerus fuerza * 2;

// Declaración de texto
esto nombre : textum "Somos la resistencia";

// Declaración de decimal
esto gravedad : decimalis 9.81;

// Declaración de un carácter
esto inicial : littera 'a'

// Declaracion de un booleano
```

Arreglos

```
// Declaración de un array de enteros
series mis_enteros[2] : numerus {1, 1}; // Array con valores iniciales
series mis_enteros_[2] : numerus; // Array de enteros sin inicialización

// Declaración de texto
series nombres[2] : textum {"Hola", "Adios"};
series nombres_[2] : textum;
```

Los arreglos pueden ser de cualquier tipo primario (textum, decimalis, numerus, etc) así como de cualquier estructura definida por el usuario o incluso de un objeto.



Asignaciones

```
esto mi_entero : numerus 10; // Declaración con un entero con valor 10
mi_entero = 23; // Asignación a mi_entero con valor 23

esto mi_booleano : falsus; // Declaración con un booleano con valor falso
mi_booleano = verum || 1 = 1; // Asignación a mi_booleano

esto mi_cadena : textum "Nothing here";
mi_cadena = "Abajo el imperio porcino " + 100 + " .. no tenemos miedo";

esto mi_float : decimalis 12.4;
mi_float = 17 / 2 ;
```

Arreglos

```
nombres[0] = "Capitán Espárragos";

nombres[1] = nombres[0] + " clon";
```

Uso de estructuras

Todas las estructuras deberían venir ya declaradas en un archivo .y

El uso de las estructuras cambia ligeramente al uso de estructuras en de la misión anterior.

Estructuras y Estructuras Anidadas

En el archivo .y



```
estructura Direccion:
    cadena calle
    entero numero

estructura Persona:
    cadena nombre
    entero edad
    Direccion domicilio // Estructura anidada

estructura Estudiante:
    cadena nombre
    entero calificaciones[3] // Un array dentro de la estructura
```

En el archivo .pig

```
esto mi_direccion : Direccion {"Calle Real", 42};
esto ciudadano : Persona {"Valeria", 25, {"Avenida Central", 500}};
esto ciudadano2 : Persona { miObjeto.getNombre(), 12, mi_direccion };
series resistencia[3] : Persona; // Un array de 3 elementos de tipo Persona

series mis_enteros[3] : numerus {1, 1, 1};
esto alumno_ejemplo : Estudiante {"Carlos", mis_enteros };
```



Uso de objetos

```
esto miObjeto : novus Persona(12, "Profesor");  
esto otroObjeto : novus Persona(12, miObjeto, novus Persona());  
series misObjetos[10] : Persona;  
  
miObjeto.nombre = "Yennifer"  
misObjetos[9].hablar(miObjeto.getNombre());  
miCadena = miObjeto.apellidos[0].getNombre();
```

Uso de funciones

Las funciones que no esten relacionadas a un objeto, tienen necesariamente que venir especificadas en algún archivo .y

Condicionales

```
si (x > 10 && y < 5) {  
    // si se cumple la condición  
} finis;
```

```
si (x > 10 && y < 5) {  
    // si se cumple la condición  
} aliter {  
    // si no se cumple la condición  
} finis;
```

```
si (x > 10 && y < 5) {  
    // si se cumple la condición
```



```
} aliter (x > 10) {  
    // si no se cumple la primera condición pero si la segunda  
} aliter {  
    // si no se cumple ninguna condición  
} finis;
```

Ciclos

Ciclo simple

```
dum (x < 100) {  
    // Ejecutar mientras x sea menor a 100  
    x = x + 1;  
} finis;
```

Ciclo do while

```
facere {  
    // Táctica: ejecutar al menos una vez  
} dum (x < 10);
```

Ciclo con iterador

```
per (esto i : numerus 0; i < 10; i++) {  
    // Ejecutar 10 veces con iterador  
}
```



Interrupción de ciclos

Se pueden usar las palabras reservadas **perge** (continuar/continue) o **interrumpe** (interrumpir/break) únicamente dentro de los ciclos para saltarse instrucciones o interrumpirlos.

Funciones especiales

Estas funciones no deben declararse, se consideran parte del sistema.

Leer texto en consola:

```
<< // solo lee el texto, no lo guarda en ningun lado  
mi_textum << // lee el texto y lo guarda en una variable mi_textum
```

Imprimir:

```
>> "Holaaaaa";  
>> mi_string;  
// imprime el contenido de dos variables, pueden imprimirse cuantas sean  
>> mi_string >> mi_textum;
```



Características obligatorias de la práctica

- ☐ Análisis léxico y sintáctico del lenguaje, con el manejo adecuado de errores léxicos, sintácticos y semánticos.
- ☐ Interfaz amigable para el usuario
- ☐ Coloreado del código (los colores quedan a libertad del desarrollador) para los tres lenguajes y en tiempo real. No se permiten librerías para el coloreado.
- ☐ Gestión del árbol de trabajo (abrir, guardar y descargar, tanto carpetas como archivos)
- ☐ Documentación técnica
 - ☐ Tecnologías utilizadas
 - ☐ Diagrama de clases
 - ☐ Especificaciones de palabras reservadas, símbolos y gramática
 - ☐ Tabla de compatibilidad de tipos (a qué tipo se convierte si se suma un string con un int por ejemplo) para los tres lenguajes.
- ☐ Manual de usuario
- ☐ Cuartetas convertidas en bytecode ejecutable en un archivo que se pueda compilar en c o sus variantes.



Consideraciones importantes

- ☐ Proyecto obligatorio para tener derecho al Proyecto 2
- ☐ Realizar cualquier análisis sintáctico y semántico con ANTLR4
- ☐ Lenguaje de programación a utilizar: JAVA
- ☐ Si se va a utilizar código de internet entenderlo para que sea tomado como válido
- ☐ Copias obtendrán nota de 0

Entregables

- ☐ Código en Github
- ☐ Manuales técnico y de usuarios en el repositorio

Fecha de entrega xx de xxxx mediante la plataforma classroom.